

Machine Learning Series

1 What is Machine Learning and Why It Matters: Everything You Need to Know

2 Machine Learning Algorithms: (With Essentials, Principles, Types & Examples covered)

3 Overfitting and Underfitting in Machine Learning + [Example]

4 What is Bias-Variance Tradeoff in Machine Learning

Develop Job Ready Digital Skills with Our AI & Data Programs

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

Share This Article

knowledgehut

rad

All Blogs

Home

Blog

Data Science

Bagging and Random Forest in Ma...

Ready to Master the Skills that Drive Your Career?

Avail your free 1:1 mentorship session.

First name*

Last name*

Email*

+91

Phone*

Your Message (Optional)

+

I want to receive updates directly on Whatsapp.

Talk to Us

Join Our Community

Get Benefits of Course offers

Join Telegram

Bagging and Random Forest in Machine Learning

arkar

Published05th Sep, 2023

Views9,970

Read Time8 Mins

1 article

What is Ensemble Learning?

Ensemble Techniques

Ensemble Methods

4. Why use ensemble models?

Develop Job Ready Digital Skills with Our AI & Data Programs

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

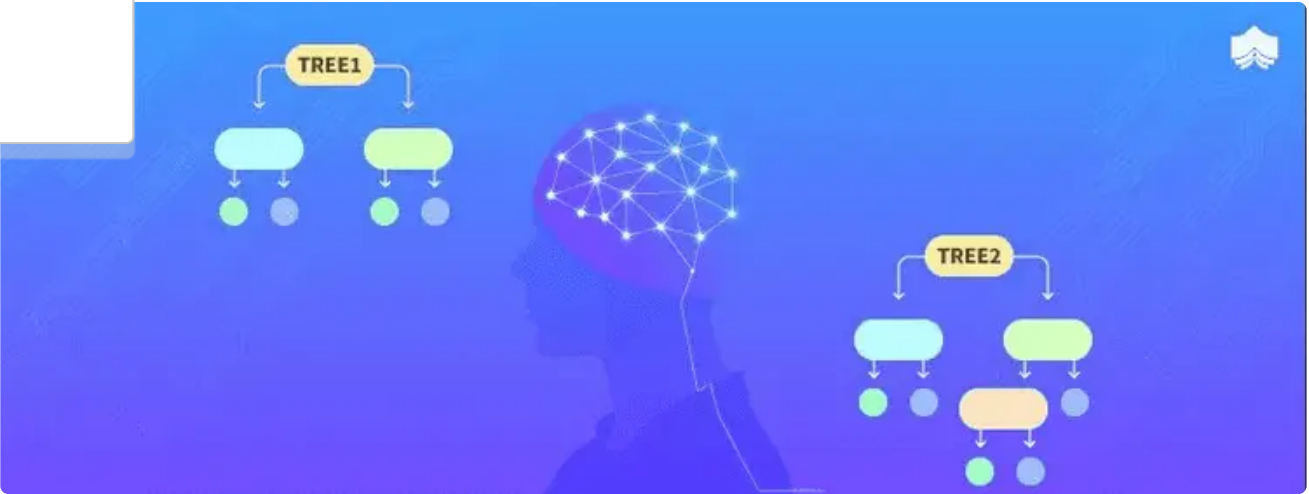
96

97

98

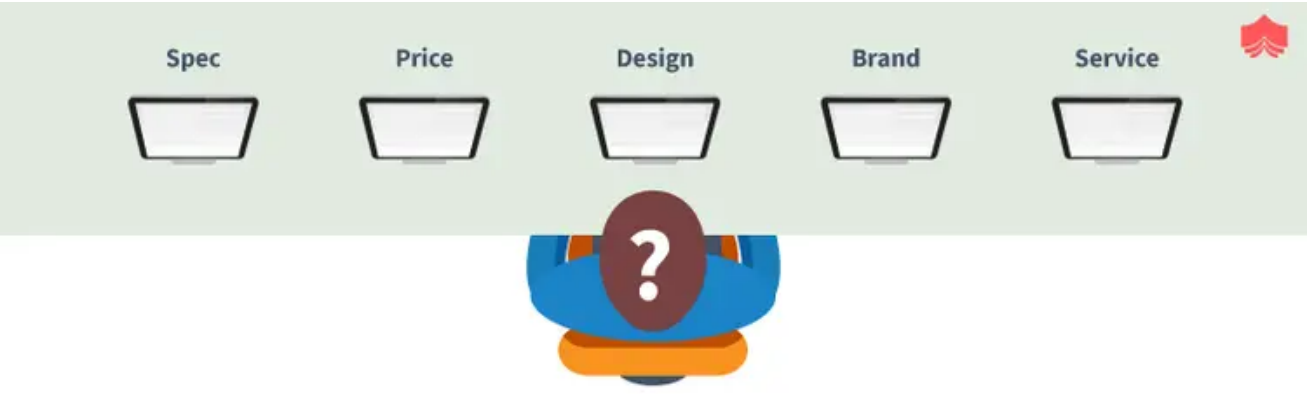
99

100



In today's world, innovations happen on a daily basis, rendering all the previous versions of that product, service, or skill set outdated and obsolete. In such a dynamic and chaotic space, how can we make an informed decision without getting carried away by plain hype? To make the right decisions, we can take a course on [machine learning with Python projects](#) and learn to follow a set of processes; investigate the current scenario, chart down your expectations, collect reviews from others, explore your options, select the best solution after weighing the pros and cons, make a decision and take the requisite action.

For example, if you are looking to purchase a computer, will you simply walk up to the store and pick any laptop or notebook? It's highly unlikely that you would do so. You would probably search on Amazon, browse a few web portals where people have posted their reviews, and compare different models, checking for their features, specifications, and prices. You will also probably ask your friends and colleagues for their opinion. In short, you would not directly jump to a conclusion, but will instead make a decision considering the opinions and reviews of other people as well.



Ensemble models in machine learning also operate in a similar manner. They combine the decisions from multiple models to improve the overall performance. The objective of this article is to introduce the concept of ensemble learning and understand algorithms like bagging and random forest which use a similar technique.

If you are inspired by the opportunities provided by machine learning, enroll in our [data science with python training](#) courses.

What is Ensemble Learning?

Ensemble methods aim at improving the predictive performance of a given statistical learning or model fitting technique. The general principle of ensemble methods is to construct a linear combination of some model fitting method, instead of using a single fit of the method.

An ensemble is itself a supervised learning algorithm because it can be trained and then used to make predictions. Ensemble methods combine several decision trees classifiers to produce better predictive performance than a single decision tree classifier. The main principle behind the ensemble model is that a group of weak learners come together to form a strong learner, thus increasing the accuracy of the model. When we try to predict the target variable using any machine learning technique, the main causes of the difference in actual and predicted values are **noise, variance, and bias**. Ensemble helps to reduce these factors (except noise, which is irreducible error). The noise-related error is mainly due to noise in the training data and can't be removed. However, the errors due to bias and variance can be reduced. The total error can be expressed as follows:

Total Error = Bias + Variance + Irreducible Error

A measure such as **mean square error (MSE)** captures all of these errors for a continuous target variable and can be represented as follows:

MSE = E[(Y - f(x))^2]

Where, E stands for the expected mean, Y represents the actual target values and f(x) is the predicted values for the target variable. It can be broken down into its components such as bias, variance and noise as shown in the following formula:

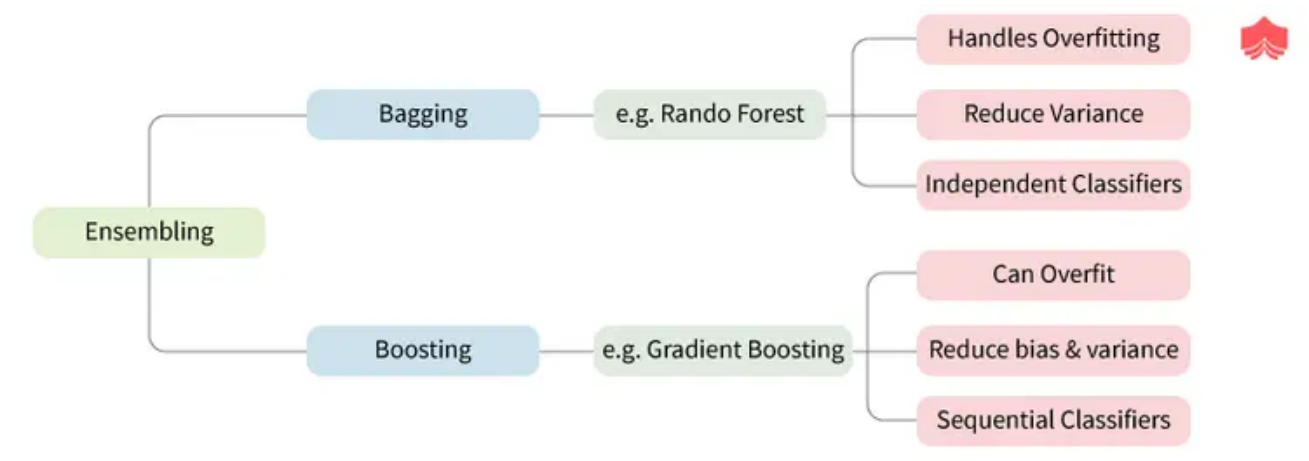
MSE = [E[f(x)] - f(x)]^2 + E[(f(x) - E[f(x)])^2] + ε

Bias Variance Noise

Using techniques like Bagging and Boosting helps to decrease the variance and increase the robustness of the model. Combinations of multiple classifiers decrease variance, especially in the case of unstable classifiers, and may produce a more reliable classification than a single classifier.

Ensemble Algorithm

The goal of ensemble algorithms is to combine the predictions of several base estimators built with a given learning algorithm in order to improve generalizability / robustness over a single estimator.



There are two families of ensemble methods which are usually distinguished:

- 1. **Averaging methods.** The driving principle is to build several estimators independently and then to average their predictions. On average, the combined estimator is usually better than any of the single base estimator because its variance is reduced. | **Examples:** Bagging methods, Forests of randomized trees.
- 2. **Boosting methods.** Base estimators are built sequentially and one tries to reduce the bias of the combined estimator. The motivation is to combine several weak models to produce a powerful ensemble. **Examples:** AdaBoost, Gradient Tree Boosting.

Advantages of Ensemble Algorithm

- Ensemble is a proven method for improving the accuracy of the model and works in most of the cases.
- Ensemble makes the model more robust and stable thus ensuring decent performance on the test cases in most scenarios.
- You can use ensemble to capture linear and simple as well nonlinear complex relationships in the data. This can be done by using two different models and forming an ensemble of two.

Disadvantages of Ensemble Algorithm

- Ensemble reduces the model interpret-ability and makes it very difficult to draw any crucial business insights at the end
- It is time-consuming and thus might not be the best idea for real-time applications
- The selection of models for creating an ensemble is an art which is really hard to master

Basic Ensemble Techniques

- **Max Voting:** Max-voting is one of the simplest ways of combining predictions from multiple machine learning algorithms. Each base model makes a prediction and votes for each sample. The sample class with the highest votes is considered in the final predictive class. It is mainly used for classification problems.
- **Averaging:** Averaging can be used while estimating the probabilities in classification tasks. But it is usually used for regression problems. Predictions are extracted from multiple models and an average of the predictions are used to make the final prediction.
- **Weighted Average:** Like averaging, weighted averaging is also used for regression tasks. Alternatively, it can be used while estimating probabilities in classification problems. Base learners are assigned different weights, which represent the importance of each model in the prediction.

Machine Learning Series

1

What is Machine Learning and Why It Matters: Everything You Need to Know

2

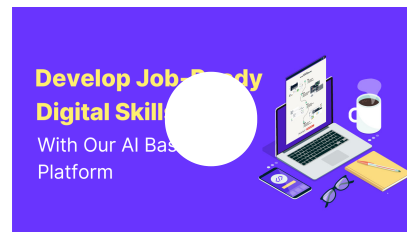
Machine Learning Algorithms: (With Essentials, Principles, Types & Examples covered)

3

Overfitting and Underfitting in Machine Learning + [Example]

4

What is Bias-Variance Tradeoff in Machine Learning



Share This Article

Ready to Master the Skills that Drive Your Career?

Avail your free 1:1 mentorship session.

First name*

Last name*

Email*

+91

Phone*

Your Message (Optional)

+

I want to receive updates directly on Whatsapp.

Join Our Community

Get Benefits of course offers

Join Telegram

Ensemble methods became popular as a relatively simple device to improve the predictive performance of a base procedure. There are different reasons for this: the bagging procedure turns out to be a variance reduction scheme, at least for some base procedures. On the other hand, boosting methods are primarily reducing the bias of the base procedure. This already indicates that bagging and boosting are different ensemble methods. From the perspective of prediction, random forest is about as good as boosting, and often better than bagging.

Boosting Aggregation or Bagging tries to implement similar learners on small populations and then takes a mean of all the predictions.

Boosting combines Bootstrapping and Aggregation to form one ensemble model which reduces the variance error and helps to avoid overfitting.

Boosting algorithms include:

- AdaBoost
- GBM
- XGBM
- Light GBM
- CatBoost

Why use ensemble models?

Ensemble models help in improving algorithm accuracy as well as the robustness of a model. Both Bagging and Boosting should be known by data scientists and machine learning engineers and especially people who are planning to attend data science/machine learning interviews.

Ensemble learning uses hundreds to thousands of models of the same algorithm and then work hand in hand to find the correct classification. You may also consider the [fable of the blind men and the elephant](#) to understand ensemble learning, where each blind man found a feature of the elephant and they all thought it was something different. However, if they would work together and discussed among themselves, they might have figured out what it is.

Using techniques like bagging and boosting leads to increased robustness of statistical models and decreased variance. Now the question becomes, between these different "B" words. Which is better?

Which is better, Bagging or Boosting?

There is no perfectly correct answer to that. It depends on the data, the simulation and the circumstances.

Bagging and boosting decrease the variance of your single estimate as they combine several estimates from different models. So the result may be a model with **higher stability**.

If the problem is that the single model gets a very low performance, Bagging will rarely get a **better bias**. However, Boosting could generate a combined model with lower errors as it optimizes the advantages and reduces pitfalls of the single model.

By contrast, if the difficulty of the single model is **overfitting**, then Bagging is the best option. Boosting for its part doesn't help to avoid over-fitting; in fact, this technique is faced with this problem itself. For this reason, Bagging is effective more often than boosting. In this article we will discuss about Bagging, we will cover Boosting in the next post. But first, let us look into the very important concept of bootstrapping.

Bootstrap Sampling

Sampling is the process of selecting a subset of observations from the population with the purpose of estimating some parameters about the whole population. [Resampling methods](#), on the other hand, are used to improve the estimates of the population parameters.

In machine learning, the bootstrap method refers to random sampling with replacement. This sample is referred to as a resample. This allows the model or algorithm to get a better understanding of the various biases, variances and features that exist in the resample. Taking a sample of the data allows the resample to contain different characteristics then it might have contained as a whole. This is demonstrated in figure 1 where each sample population has different pieces, and none are identical. This would then affect the overall mean, [standard deviation](#) and other descriptive metrics of a data set. In turn, it can develop more robust models.

Bootstrapping is also great for small size data sets that can have a tendency to overfit. In fact, we recommended this to one company that was concerned because their data sets were far from "Big Data". Bootstrapping can be a solution in this case because algorithms that utilize bootstrapping can be more robust and handle new data sets depending on the methodology chosen (boosting or bagging).

The reason behind using the bootstrap method is because it can test the stability of a solution. By using multiple sample data sets and then testing multiple models, it can increase robustness. Perhaps one sample data set has a larger mean than another, or a different standard deviation. This might break a model that was overfit, and not tested using data sets with different variations.

One of the many reasons bootstrapping has become very common is because of the increase in computing power. This allows for many times more permutations to be done with different resamples than previously. Bootstrapping is used in both Bagging and Boosting

Let us assume we have a sample of 'n' values (x) and we'd like to get an estimate of the mean of the sample.

mean(x) = 1/n * sum(x)

Consider a sample of 100 values (x) and we'd like to get an estimate of the mean of the sample. We can calculate the mean directly from the sample as:

We know that our sample is small and that the mean has an error in it. We can improve the estimate of our mean using the bootstrap procedure:

1. Create many (e.g., 1000) random sub-samples of the data set with replacement (meaning we can select the same value multiple times).
2. Calculate the mean of each sub-sample.

Share This Article

3. Calculate the average of all of our collected means and use that as our estimated mean for the data.

Example: Suppose we used 3 re-samples and got the mean values 2.3, 4.5 and 3.3.

The average of these we could take the estimated mean of the data to be 3.43. This process can be used to estimate other quantities like the standard deviation and even quantities used in machine learning algorithms, like learned weights.

Using Python, we do not have to implement the bootstrap method manually. The sklearn library provides an implementation that creates a single bootstrap sample from a dataset.

The `sklearn.utils.resample()` scikit-learn function can be used for sampling. It takes as input the data array, whether or not to sample with replacement, the size of the sample, and the seed for the pseudorandom number generator used prior to the sampling.

For example, let us create a bootstrap that creates a sample with replacement with 4 observations and uses a value of 1 for the pseudorandom number generator.

```
resample(data, replace=True, n_samples=4, random_state=1)
```

Copy Code

The sklearn bootstrap API does not allow to easily gather the out-of-bag observations that could be used as a test set to evaluate a fit model, in the univariate case we can gather the out-of-bag observations using a simple Python list comprehension.

```
# out of bag observations
oob = [x for x in data if x not in boot]
```

Copy Code

Let us look at a small example and execute it.

```
# scikit-learn bootstrap
from sklearn.utils import resample
# data sample
data = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]
# prepare bootstrap sample
boot = resample(data, replace=True, n_samples=4, random_state=1)
print('Bootstrap Sample: %s' % boot)
# out of bag observations
oob = [x for x in data if x not in boot]
print('OOB Sample: %s' % oob)
```

Copy Code

The output will include the observations in the bootstrap sample and those observations in the out-of-bag sample.

```
Bootstrap Sample: [0.6, 0.4, 0.5, 0.1]
OOB Sample: [0.2, 0.3]
```

Copy Code

Bagging

Bootstrap Aggregation, also known as **Bagging**, is a powerful ensemble method that was proposed by Leo Breiman in 1994 to prevent overfitting. The concept behind bagging is to combine the predictions of several base learners to create a more accurate output. Bagging is the application of the Bootstrap procedure to a high-variance machine learning algorithm, typically decision trees.

1. Suppose there are N observations and M features. A sample from observation is selected randomly with replacement (Bootstrapping).
2. A subset of features are selected to create a model with sample of observations and subset of features.
3. Feature from the subset is selected which gives the best split on the training data.
4. This is repeated to create many models and every model is trained in parallel.
5. Prediction is given based on the aggregation of predictions from all the models.

This approach can be used with machine learning algorithms that have a high variance, such as decision trees. A separate model is trained on each bootstrap sample of data and the average output of those models used to make predictions. This technique is called bootstrap aggregation or bagging for short.

Variance means that an algorithm's performance is sensitive to the training data, with high variance suggesting that the more the training data is changed, the more the performance of the algorithm will vary.

The performance of high variance machine learning algorithms like unpruned decision trees can be improved by training many trees and taking the average of their predictions. Results are often better than a single decision tree.

What Bagging does is help reduce variance from models that are might be very accurate, but only on the data they were trained on. This is also known as overfitting.

Overfitting is when a function fits the data too well. Typically this is because the actual equation is much too complicated to take into account each data point and outlier.

Bagging gets around this by creating its own variance amongst the data by sampling and replacing data while it tests multiple hypothesis(models). In turn, this reduces the noise by utilizing multiple samples that would most likely be made up of data with various attributes(median, average, etc).

Once each model has developed a hypothesis. The models use *voting for classification or averaging for regression*. This is where the "Aggregating" in "Bootstrap Aggregating" comes into play. Each hypothesis has the same weight as all the others. *When we later discuss boosting, this is one of the places the two methodologies differ.*

Essentially, all these models run at the same time, and vote on which hypothesis is the most accurate.

This helps to decrease variance i.e. reduce the overfit.

Advantages

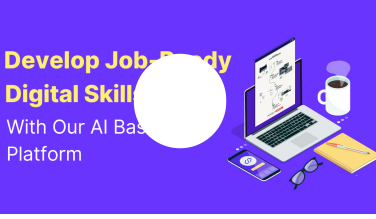
- Bagging takes advantage of ensemble learning wherein multiple weak learners outperform a single strong learner.
- It helps reduce variance and thus helps us avoid overfitting.

Disadvantages

Get a 1:1 Mentorship call with our Career Advisor Book free session

Machine Learning Series

- 1 What is Machine Learning and Why It Matters: Everything You Need to Know
- 2 Machine Learning Algorithms: (With Essentials, Principles, Types & Examples covered)
- 3 Overfitting and Underfitting in Machine Learning + [Example]
- 4 What is Bias-Variance Tradeoff in Machine Learning



Machine Learning Series

1

What is Machine Learning and Why It Matters: Everything You Need to Know

2

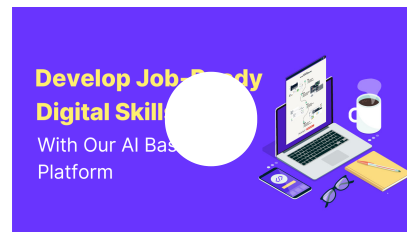
Machine Learning Algorithms: (With Essentials, Principles, Types & Examples covered)

3

Overfitting and Underfitting in Machine Learning + [Example]

4

What is Bias-Variance Tradeoff in Machine Learning



Share This Article

Ready to Master the Skills that Drive Your Career?

Avail your free 1:1 mentorship session.

First name*

Last name*

Email*

+91

Phone*

Your Message (Optional)

+

I want to receive updates directly on Whatsapp.

Join Our Community

Get Benefits of Course offers

Join Telegram

test sets

ap Aggregating Trees

We have already discussed bootstrap aggregating (or bagging), we can create an ensemble (forest) of trees where multiple training sets are generated with replacement, meaning data instances. Once the training sets are created, a CART model can be trained on each subsample.

Features of Bagged Trees

- Reduces variance by averaging the ensemble's results.
- The resulting model uses the entire feature space when considering node splits.
- Bagging trees allow the trees to grow without pruning, reducing the tree-depth sizes and resulting in high variance but lower bias, which can help improve predictive power.

Limitations to Bagging Trees

The main limitation of bagging trees is that it uses the entire feature space when creating splits in the trees. Suppose some variables within the feature space are indicating certain predictions, there is a risk of having a forest of correlated trees, which actually increases bias and reduces variance.

Why a Forest is better than One Tree?

The main objective of a machine learning model is to generalize properly to new and unseen data. When we have a flexible model, **overfitting** takes place. A flexible model is said to have high **variance** because the learned parameters (such as the structure of the decision tree) will vary with the training data.

On the other hand, an inflexible model is said to have high **bias** as it makes **assumptions** about the training data. An inflexible model may not have the capacity to fit even the training data and in both cases — high variance and high bias — the model is not able to generalize new and unseen data properly.

You can through the article on one of the foundational concepts in machine learning, [bias-variance tradeoff](#) which will help you understand that the balance between creating a model that is so flexible memorizes the training data and an inflexible model cannot learn the training data.

The main reason why the decision tree is prone to overfitting when we do not limit the maximum depth is because it has unlimited flexibility, which means it keeps growing unless there is one leaf node for every single observation.

Instead of limiting the depth of the tree which results in reduced variance and increase in bias, we can combine many decision trees into a single ensemble model known as the **random forest**.

What is Random Forest algorithm?

Random forest is like bootstrapping algorithm with Decision tree (CART) model. Suppose we have 1000 observations in the complete population with 10 variables. Random forest will try to build multiple CART along with different samples and different initial variables. It will take a random sample of 100 observations and then chose 5 initial variables randomly to build a CART model. It will go on repeating the process say about 10 times and then make a final prediction on each of the observations. Final prediction is a function of each prediction. This final prediction can simply be the mean of each prediction.

The random forest is a model made up of many decision trees. Rather than just simply averaging the prediction of trees (which we could call a "forest"), this model uses two key concepts that gives it the name *random*:

1. Random sampling of training data points when building trees
2. Random subsets of features considered when splitting nodes

How the Random Forest Algorithm Works

The basic steps involved in performing the random forest algorithm are mentioned below:

1. Pick N random records from the dataset.
2. Build a decision tree based on these N records.
3. Choose the number of trees you want in your algorithm and repeat steps 1 and 2.
4. In case of a regression problem, for a new record, each tree in the forest predicts a value for Y (output). The final value can be calculated by taking the average of all the values predicted by all the trees in the forest. Or, in the case of a classification problem, each tree in the forest predicts the category to which the new record belongs. Finally, the new record is assigned to the category that wins the majority vote.

Using Random Forest for Regression

Here we have a problem where we have to predict the gas consumption (in millions of gallons) in 48 US states based on petrol tax (in cents), per capita income (dollars), paved highways (in miles) and the proportion of population with the driving license. We will use the random forest algorithm via the Scikit-Learn Python library to solve this regression problem.

First we import the necessary libraries and our dataset.

```
import pandas as pd
import numpy as np
dataset = pd.read_csv('/content/petrol_consumption.csv')
dataset.head()
```

Copy Code

	Petrol_tax	Average_income	paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.0	3571	1976	0.525	541
1	9.0	4092	1250	0.572	524
2	9.0	3865	1586	0.580	561
3	7.5	4870	2351	0.529	414
4	8.0	4399	431	0.544	410

You will notice that the values in our dataset are not very well scaled. Let us scale them down before training the algorithm.

Preparing Data For Training

We will perform two tasks in order to prepare the data. Firstly we will divide the data into 'attributes' and 'label' sets. The resultant will then be divided into training and test sets.

```
X = dataset.iloc[:, 0:4].values
y = dataset.iloc[:, 4].values
```

Copy Code

Now let us divide the data into training and testing sets:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=0)
```

Copy Code

Feature Scaling

The dataset is not yet a scaled value as you will see that the Average_Income field has values in the range of thousands while Petrol_tax has values in the range of tens. It will be better if we scale our data. We will use Scikit-Learn's StandardScaler class to do the same.

```
# Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Copy Code

Training the Algorithm

Now that we have scaled our dataset, let us train the random forest algorithm to

Machine Learning Series

1

What is Machine Learning and Why It Matters: Everything You Need to Know

2

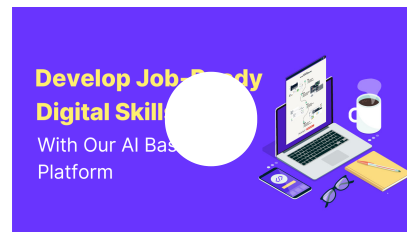
Machine Learning Algorithms: (With Essentials, Principles, Types & Examples covered)

3

Overfitting and Underfitting in Machine Learning + [Example]

4

What is Bias-Variance Tradeoff in Machine Learning



Share This Article

Ready to Master the Skills that Drive Your Career?

Avail your free 1:1 mentorship session.

First name*

Last name*

Email*

+91

Phone*

Your Message (Optional)

+

I want to receive updates directly on Whatsapp.

Join Our Community

Get Benefits of course offers

Join Telegram

```
from sklearn.ensemble import Random Forest Regressor
regressor = Random Forest Regressor(n_estimators=20,random_state=0)
regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)
```

Copy Code

omForestRegressor is used to solve regression problems via random forest. t important parameter of the RandomForestRegressor class is the itors parameter. This parameter defines the number of trees in the random ere we started with n_estimator=20 and check the performance of the n. You can find details for all of the parameters of RandomForestRegressor

Evaluating the Algorithm

valuate the performance of the algorithm. For regression problems the sed to evaluate an algorithm are mean absolute error, mean squared error, mean squared error.

```
from sklearn import metrics
print('Mean Absolute Error:', metrics.mean_absolute_error(y_test, y_pred))
print('Mean Squared Error:', metrics.mean_squared_error(y_test, y_pred))
print('Root Mean Squared Error:', metrics.mean_squared_error(y_test, y_pred))
```

Copy Code

Mean Absolute Error: 51.76500000000001
Mean Squared Error: 4216.166749999999
Root Mean Squared Error: 64.93201637097064

Copy Code

With 20 trees, the root mean squared error is 64.93 which is greater than 10 percent of the average petrol consumption i.e. 576.77. This may indicate, among other things, that we have not used enough estimators (trees).

Let us now change the number of estimators to 200, the results are as follows:

Mean Absolute Error: 48.338999999999999
Mean Squared Error: 3494.2330150000003
Root Mean Squared Error: 59.112037818020234

Copy Code

The graph below shows the decrease in the value of the root mean squared error (RMSE) with respect to number of estimators.

You will notice that the error values decrease with the increase in the number of estimators. You may consider 200 a good number for n_estimators as the rate of decrease in error diminishes. You may try playing around with other parameters to figure out a better result.

Using Random Forest for Classification

Now let us consider a classification problem to predict whether a bank currency note is authentic or not based on four attributes i.e. variance of the image wavelet transformed image, skewness, entropy, andkurtosis of the image. We will use Random Forest Classifier to solve this binary classification problem. Let's get started.

```
import pandas as pd
import numpy as np
dataset = pd.read_csv('/content/bill_authentication.csv')
dataset.head()
```

Copy Code

	Variance	Skewness	Kurtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Similar to the data we used previously for the regression problem, this data is not scaled. Let us prepare the data for training.

Preparing Data For Training

The following code divides data into attributes and labels:

```
X = dataset.iloc[:, 0:4].values
y = dataset.iloc[:, 4].values
```

Copy Code

The following code divides data into training and testing sets:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

Copy Code

Feature Scaling

We will do the same thing as we did for the previous problem.

```
# Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Copy Code

Training the Algorithm

Now that we have scaled our dataset, let us train the random forest algorithm to solve this classification problem.

```
from sklearn.ensemble import Random Forest Classifier
classifier = RandomForestClassifier(n_estimators=20, random_state=0)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

Copy Code

For classification, we have used RandomForestClassifier class of the sklearn.ensemble library. It takes n_estimators as a parameter. This parameter defines the number of trees in out random forest. Similar to the regression problem, we have started with 20 trees here. You can find details for all of the parameters of Random Forest Classifier here.

Evaluating the Algorithm

For evaluating classification problems, the metrics used are accuracy, confusion matrix, precision recall, and F1 values

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
print(confusion_matrix(y_test,y_pred))
print(classification_report(y_test,y_pred))
print(accuracy_score(y_test, y_pred))
```

Copy Code

Machine Learning Series

- 1 What is Machine Learning and Why It Matters: Everything You Need to Know
- 2 Machine Learning Algorithms: (With Essentials, Principles, Types & Examples covered)
- 3 Overfitting and Underfitting in Machine Learning + [Example]
- 4 What is Bias-Variance Tradeoff in Machine Learning



Share This Article

The output will look something like this:

Output:

Ready to Master the Skills that Drive Your Career?

Avail your free 1:1 mentorship session.

First name*

Last name*

Email*

+91

Phone*

Your Message (Optional)

+

I want to receive updates directly on Whatsapp.

Copy Code

2]

117]

	Precision	recall	f1-score	support
	0.99	0.99	0.99	157
	0.98	0.99	0.99	118
			0.99	275
g	0.99	0.99	0.99	275
9090909091	0.99	0.99	0.99	275

Unlike the regression problem, changing the number of estimators for this problem make any difference in the results.

Join Our Community

Get Benefits of Course offers

Join Telegram

An accuracy of 98.9% is pretty good. In this case, we have seen that there is not much improvement if the number of trees are increased. You may try playing around with other parameters of the RandomForestClassifier class and see if you can improve on our results.

Advantages and Disadvantages of using Random Forest

As with any algorithm, there are advantages and disadvantages to using it. Let us look into the pros and cons of using Random Forest for classification and regression.

Advantages

- Random forest algorithm is unbiased as there are multiple trees and each tree is trained on a subset of data.
- Random Forest algorithm is very stable. Introducing a new data in the dataset does not affect much as the new data impacts one tree and is pretty hard to impact all the trees.
- The random forest algorithm works well when you have both categorical and numerical features.
- With missing values in the dataset, the random forest algorithm performs very well.

Disadvantages

- A major disadvantage of random forests lies in their complexity. More computational resources are required and also results in the large number of decision trees joined together.
- Due to their complexity, training time is more compared to other algorithms.

Summary

In this article, we have covered what ensemble learning is and discussed basic ensemble techniques. We also looked into bootstrap sampling involves iteratively resampling of a dataset with a replacement which allows the model or algorithm to get a better understanding of various features. Then we moved on to bagging followed by random forest. We also implemented random forest in Python for both regression and classification and came to the conclusion that increasing the number of trees or estimators does not always make a difference in a classification problem. However, in regression, there is an impact.

We have covered most of the topics related to algorithms in our series of machine learning blogs. Check out KnowledgeHut machine learning with Python projects for more lucrative career options in this landscape.

Priyankur Sarkar
Data Science Enthusiast

Priyankur Sarkar loves to play with data and get insightful results out of it, then turn those data insights and results in business growth. He is an electronics engineer with a versatile experience as an individual contributor and leading teams, and has actively worked towards building Machine Learning capabilities for organizations.

Upcoming Data Science Batches & Dates

Name	Date	Fee	Know more
Data Science Bootcamp	Cohort starts on 15 Jan 2024, Flex-Track batch	INR 72,000	View Details
Data Analyst Bootcamp	Cohort starts on 31 Jan 2024, Weekend batch	INR 79,999	View Details
Data Engineer Bootcamp	To be announced shortly	To be announced shortly	View Details
Python Programming	Cohort starts on 20 Jan 2024, Weekend batch	INR 17,999	View Details

Connect with usGet Our Weekly NewsletterWe Accept

USA: +1-469-442-0620, +1-832-684-0080	UK: +44-2036085923	Canada: +1-613-707-0763	Australia: +61-290995641
India: +91-84484-45027 Toll Free: 1800-121-9232	Singapore: +65-317-46174	New Zealand: +64-36694791	UAE: Toll Free 8000180860
	Malaysia: +601548770914	Ireland: +353-12708328	

Company

About Us
Careers
Customer Speak
Accreditation
Media
Contact Us
Grievance Redressal

Offerings

Live virtual (online)
Classroom
Agile Services
Corporate Training

Resources

Course Info
Tutorials
Blog
Interview Questions
Practice Tests
Master Classes

Partner with us

Become an Instructor
Hire from Us

Support

FAQs
Terms & Conditions
Privacy Policy & Disclaimer
Cancellation & Refund Policy
Site map

Top Categories

Agile Management Courses
Project Management Courses
IT Service Management Courses
Programming Courses
Web Development Courses
Mobile App Development Courses
Six Sigma Courses

Cloud Computing Courses
Devops Courses
Business Management Courses
Data Science Courses
BI and Visualization Courses
Quality Management Courses
Kanban Courses

Top Courses

CSM Certification
CSPO Certification
Leading SAFe 6.0 Certification
PSM Certification
PMP Certification
ITIL Foundation Certification

PRINCE2 Certification
Devops Foundation Certification
Data Science with Python Certification
Full-Stack Development Bootcamp
Front-End Development Bootcamp
Python Certification Training

Disclaimer: The content on the website and/or Platform is for informational and educational purposes only. The user of this website and/or Platform (User) should not construe any such information as legal, investment, tax, financial or any other advice. Nothing contained herein constitutes any representation, solicitation, recommendation, promotion or advertisement on behalf of KnowledgeHut and / or its Affiliates (including but not limited to its subsidiaries, associates, employees, directors, key managerial personnel, consultants, trainers, advisors). The User is solely responsible for evaluating the merits and risks associated with use of the information included as part of the content. The User agrees and covenants not to hold KnowledgeHut and its Affiliates responsible for any and all losses or damages arising from such decision made by them basis the information provided in the course and / or available on the website and/or platform. KnowledgeHut reserves the right to cancel or reschedule events in case of insufficient registrations, or if presenters cannot attend due to unforeseen circumstances. You are therefore advised to consult a KnowledgeHut agent prior to making any travel arrangements for a workshop. For more details, please refer to the [Cancellation & Refund Policy](#). CSM®, CSPO®, CSD®, CSP®, A-CSPO®, A-CSM® are registered trademarks of Scrum Alliance®. KnowledgeHut Solutions Pvt. Ltd. is a Registered Education Ally (REA) of Scrum Alliance®. PMP is a registered mark of the Project Management Institute, Inc. CAPM is a registered mark of the Project Management Institute, [in](#)[READ MORE](#)

Machine Learning Series

- 1 What is Machine Learning and Why It Matters: Everything You Need to Know
- 2 Machine Learning Algorithms: (With Essentials, Principles, Types & Examples covered)
- 3 Overfitting and Underfitting in Machine Learning + [Example]
- 4 What is Bias-Variance Tradeoff in Machine Learning



TECHNICAL SOLUTIONS PRIVATE LIMITED. All Rights Reserved

Share This Article

Ready to Master the Skills that Drive Your Career?

Avail your free 1:1 mentorship session.

First name*

Last name*

Email*

+91

Phone*

Your Message (Optional)



I want to receive updates directly on Whatsapp.

Join Our Community

Get Benefits of

Course offers



Join Telegram

[Privacy policy](#)

[Terms of service](#)